CS1200 / CSCI E-120 Lecture 13

Independent Sets and Interval Scheduling

Fall 2026

Announcements

- Pset 4 in-class reflection survey next Monday.
- Pset 5 due next Monday.
- Midterm 2 on Nov 2.
- Embedded EthiCS module next Wed (class guest - Matt Kopec)

Independent Sets: Definition

Let $G = (V, E)$ be an undirected graph.

Independent Sets: Definition

Let $G = (V, E)$ be an undirected graph. An *independent set* is a subset $S \subseteq V$ with no edge entirely in S:

$$\{u, v\} \in E \quad \Longrightarrow \quad u \notin S \text{ or } v \notin S.$$

Independent Sets: Definition

Let $G = (V, E)$ be an undirected graph. An *independent set* is a subset $S \subseteq V$ with no edge entirely in S:

$$\{u, v\} \in E \quad \Longrightarrow \quad u \notin S \text{ or } v \notin S.$$

Problem: (Max)IndependentSet

- **Input:** a graph $G = (V, E)$.
- **Output:** an independent set of *maximum size*.

Independent Sets: Definition

Let $G = (V, E)$ be an undirected graph. An *independent set* is a subset $S \subseteq V$ with no edge entirely in S:

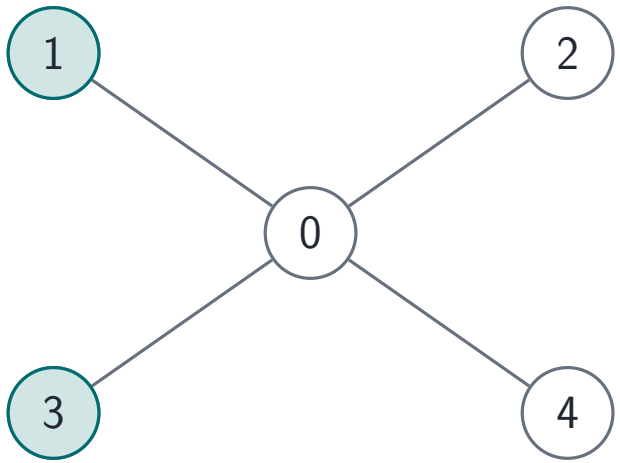
$$\{u, v\} \in E \quad \Longrightarrow \quad u \notin S \text{ or } v \notin S.$$

Problem: (Max)IndependentSet

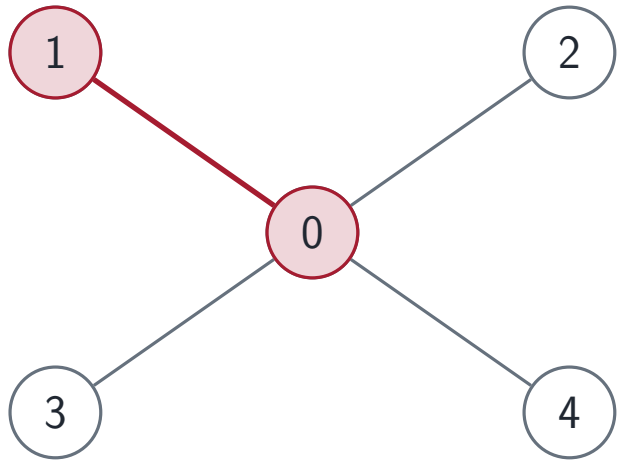
- **Input:** a graph $G = (V, E)$.
- **Output:** an independent set of *maximum size*.

In Pset 5: A notion of Maximal independent set (includes the maximum sized one, but smaller independent sets can be maximal).

Independent Sets: Example

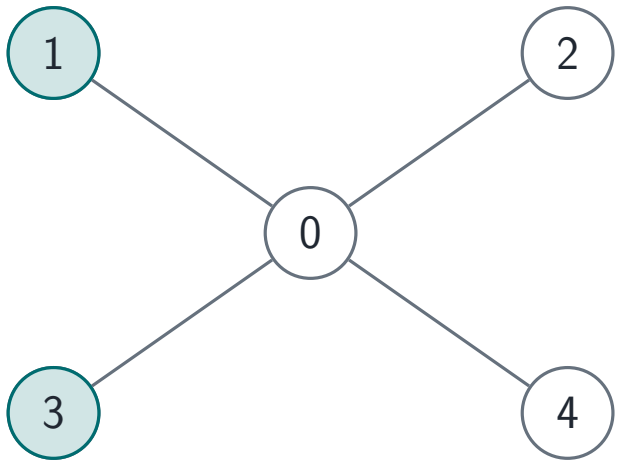


$S = \{1, 3\}$.



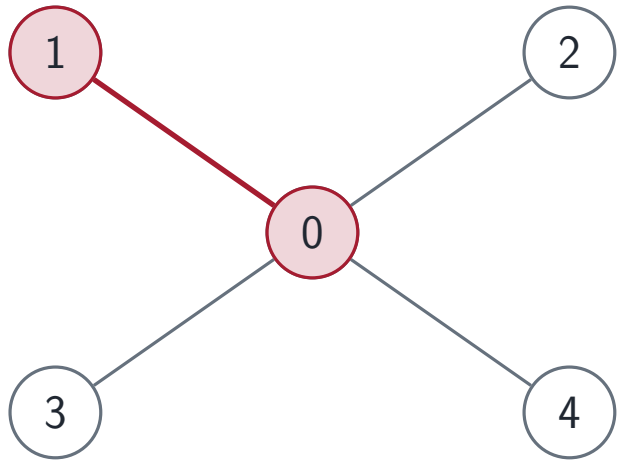
$S = \{0, 1\}$.

Independent Sets: Example



$S = \{1, 3\}$.

Independent: no edge joins its two vertices.



$S = \{0, 1\}$.

Not independent: the edge $\{0, 1\}$ lies inside S.

Real world examples

Independent sets can model almost any problem with "conflicts".

- Exam or meeting scheduling.
- Matching problems (kidney matching, dating websites, etc).
- Placing antennas on a wireless network (close by antennas with same frequency can interfere).
- Quantum computing: Rydberg blockade mechanism.

Real world examples

Independent sets can model almost any problem with "conflicts".

- Exam or meeting scheduling.
- Matching problems (kidney matching, dating websites, etc).
- Placing antennas on a wireless network (close by antennas with same frequency can interfere).
- Quantum computing: Rydberg blockade mechanism.

Main difficulty in finding largest independent set: how do you know that choosing independent set in one part of the graph optimally will not lead to sub-optimal choices later?

Greedy Algorithms

Simpler goal: Find a large independent set (need not be maximum sized).

Similar to GreedyColoring, choose an ordering over vertices and make decision to add a vertex greedily.

Choose an ordering $v_0, v_1, \ldots, v_{n-1}$
$S \leftarrow \varnothing$
for $i = 0, \ldots, n-1$ **do**
  if **then**
    $S \leftarrow S \cup \{v_i\}$
return S

Greedy Algorithms

Simpler goal: Find a large independent set (need not be maximum sized).

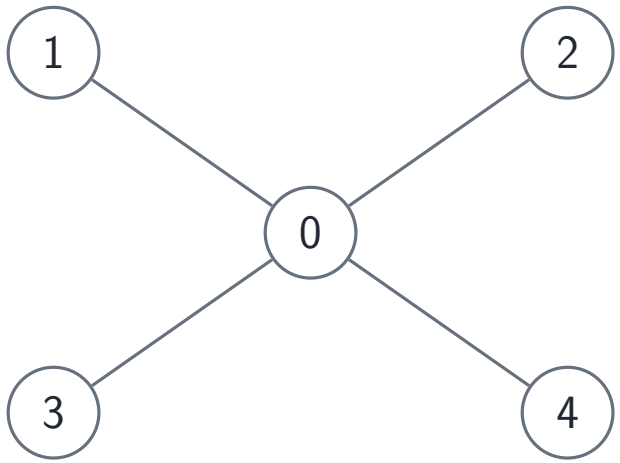
Similar to GreedyColoring, choose an ordering over vertices and make decision to add a vertex greedily.

Choose an ordering $v_0, v_1, \ldots, v_{n-1}$
$S \leftarrow \varnothing$
for $i = 0, \ldots, n-1$ **do**
    if v_i *has no neighbor in* S **then**
        $S \leftarrow S \cup \{v_i\}$
return S

Add a vertex as long as it preserves independence.

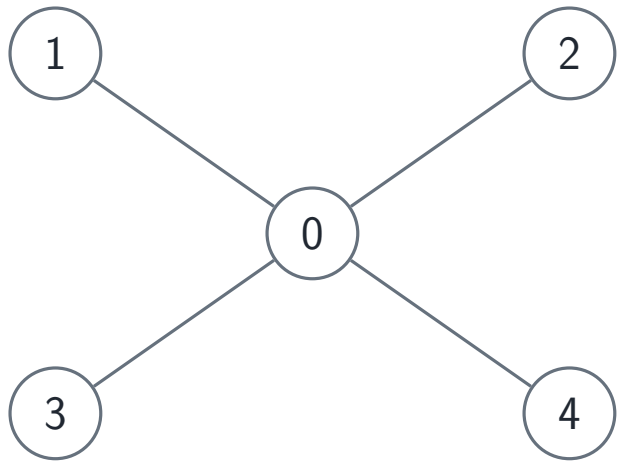
Greedy Algorithms: The Ordering Matters

Order: $0, 1, 2, 3, 4$



$S =$.

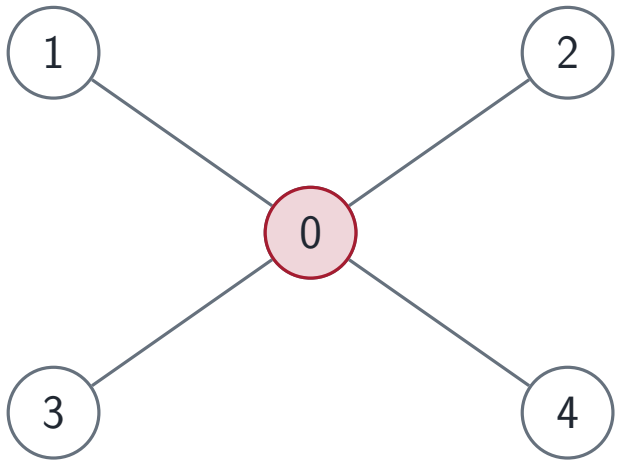
Order: $1, 2, 3, 4, 0$



$S =$.

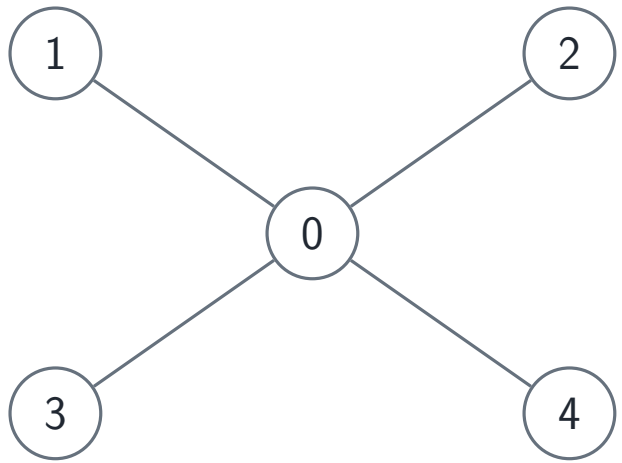
Greedy Algorithms: The Ordering Matters

Order: $0, 1, 2, 3, 4$



$S = \{0\}$.

Order: $1, 2, 3, 4, 0$

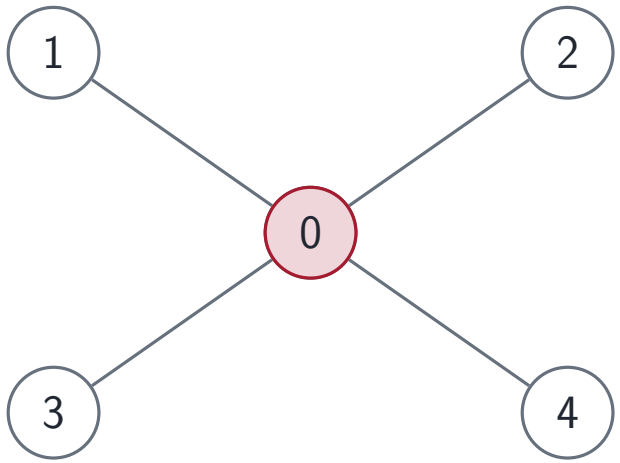


$S =$.

Every later vertex has a selected neighbor.

Greedy Algorithms: The Ordering Matters

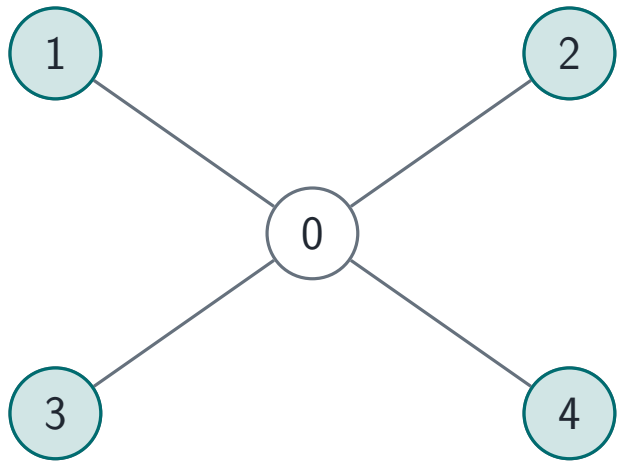
Order: $0, 1, 2, 3, 4$



$S = \{0\}$.

Every later vertex has a selected neighbor.

Order: $1, 2, 3, 4, 0$



$S = \{1, 2, 3, 4\}$.

All four leaves can be selected.

Output will always be an independent set, but need not have maximum size (as in the left figure).

Greedy Algorithms: Size Guarantee

Let $\Delta = \max_{v \in V} \deg(v)$.

Greedy Algorithms: Size Guarantee

Let $\Delta = \max_{v \in V} \deg(v)$.

Theorem. For every vertex ordering, the output of the Greedy Algorithm is an independent set S that satisfies

$$|S| \geq \frac{n}{\Delta + 1}.$$

Greedy Algorithms: Size Guarantee

Let $\Delta = \max_{v \in V} \deg(v)$.

Theorem. For every vertex ordering, the output of the Greedy Algorithm is an independent set S that satisfies

$$|S| \geq \frac{n}{\Delta + 1}.$$

Proof idea:

- Every unselected vertex has a neighbor in S; otherwise it would have been added.

Greedy Algorithms: Size Guarantee

Let $\Delta = \max_{v \in V} \deg(v)$.

Theorem. For every vertex ordering, the output of the Greedy Algorithm is an independent set S that satisfies

$$|S| \geq \frac{n}{\Delta + 1}.$$

Proof idea:

- Every unselected vertex has a neighbor in S; otherwise it would have been added.
- Each selected vertex rules out at most Δ (unselected) neighbors.

$$n - |S| \leq |S|\Delta.$$

Greedy Algorithms: Size Guarantee

Let $\Delta = \max_{v \in V} \deg(v)$.

Theorem. For every vertex ordering, the output of the Greedy Algorithm is an independent set S that satisfies

$$|S| \geq \frac{n}{\Delta + 1}.$$

Proof idea:

- Every unselected vertex has a neighbor in S; otherwise it would have been added.
- Each selected vertex rules out at most Δ (unselected) neighbors.

$$n - |S| \leq |S|\Delta.$$

For a given ordering, adjacency list representation gives $O(n + m)$ running time.

Three Variants

- **(Max) IndSet:** return a largest independent set.

Three Variants

- **(Max) IndSet:** return a largest independent set.
- **IndSet-Threshold search,** Search(G, k)**:** return an independent set of size at least k, or $\perp$ if none exists.

Three Variants

- **(Max) IndSet:** return a largest independent set.
- **IndSet-Threshold search,** Search(G, k)**:** return an independent set of size at least k, or $\perp$ if none exists.
- **IndSet-ThresholdDec,** Dec(G, k)**:** return YES or NO according to whether such a set exists.

Three Variants

- **(Max) IndSet:** return a largest independent set.
- **IndSet-Threshold search,** Search(G, k)**:** return an independent set of size at least k, or $\perp$ if none exists.
- **IndSet-ThresholdDec,** Dec(G, k)**:** return YES or NO according to whether such a set exists.

Which of these problems looks easiest to solve?

Three Variants

- **(Max) IndSet:** return a largest independent set.
- **IndSet-Threshold search,** Search(G, k)**:** return an independent set of size at least k, or $\perp$ if none exists.
- **IndSet-ThresholdDec,** Dec(G, k)**:** return YES or NO according to whether such a set exists.

Which of these problems looks easiest to solve?

These problems are all of similar difficulty! Key reason is search-to-decision reduction. In the next few slides, we will reduce these problems to each other.

IndSet-ThresholdDecision to (Max)IndSet

Given a maximum sized independent set, answer Dec(G, k).

IndSet-ThresholdDecision to (Max)IndSet

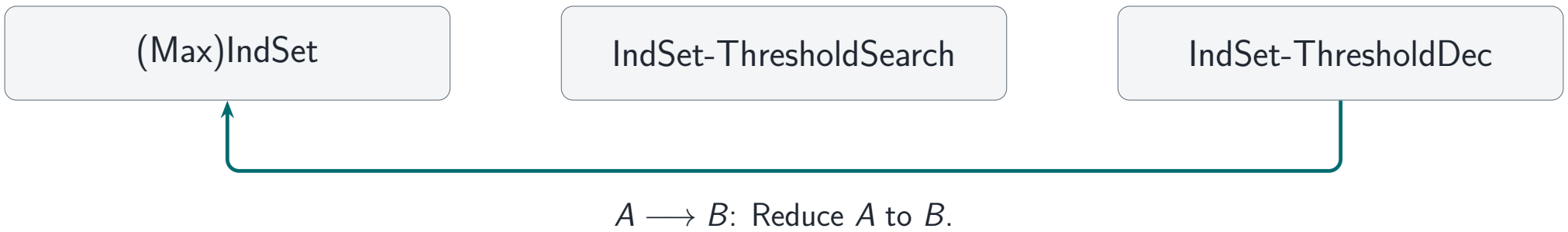
Given a maximum sized independent set, answer $\mathrm{Dec}(G, k)$.

1. Compute $S = \mathrm{Opt}(G)$ using the oracle for (Max)IndSet.
2. Return YES if $|S| \geq k$; otherwise return NO.

IndSet-ThresholdDecision to (Max)IndSet

Given a maximum sized independent set, answer $\mathrm{Dec}(G, k)$.

1. Compute $S = \mathrm{Opt}(G)$ using the oracle for (Max)IndSet.
2. Return YES if $|S| \geq k$; otherwise return NO.

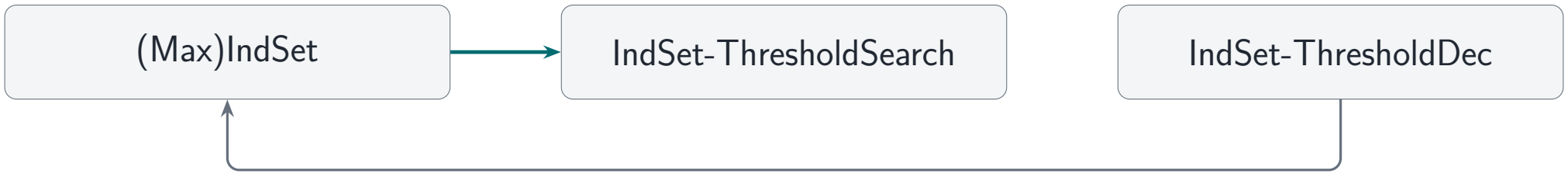


(Max)IndSet to IndSet-ThresholdSearch

Start from $k = n$ and reduce k until the oracle for IndSet-ThresholdSearch stops returning $\perp$.

(Max)IndSet to IndSet-ThresholdSearch

Start from $k = n$ and reduce k until the oracle for IndSet-ThresholdSearch stops returning $\perp$.



$A \longrightarrow B$: Reduce A to B.

IndSet-Threshold Search to IndSet-Threshold Dec

Key fact: deciding whether to include a vertex

For a vertex v, let

$$H_v = H - (\{v\} \cup N_H(v)).$$

Delete v *and all its neighbors*.

IndSet-Threshold Search to IndSet-Threshold Dec

Key fact: deciding whether to include a vertex

For a vertex v, let

$$H_v = H - (\{v\} \cup N_H(v)).$$

Delete v *and all its neighbors*.

$$H \text{ has an independent set of size at least } r \text{ containing } v$$
$$\iff \quad H_v \text{ has an independent set of size at least } r - 1.$$

IndSet-Threshold Search to IndSet-Threshold Dec

Key fact: deciding whether to include a vertex

For a vertex v, let

$$H_v = H - (\{v\} \cup N_H(v)).$$

Delete v *and all its neighbors*.

$$H \text{ has an independent set of size at least } r \text{ containing } v$$
$$\iff \quad H_v \text{ has an independent set of size at least } r - 1.$$

Proof:

IndSet-Threshold Search to IndSet-Threshold Dec

Key fact: deciding whether to include a vertex

For a vertex v, let

$$H_v = H - (\{v\} \cup N_H(v)).$$

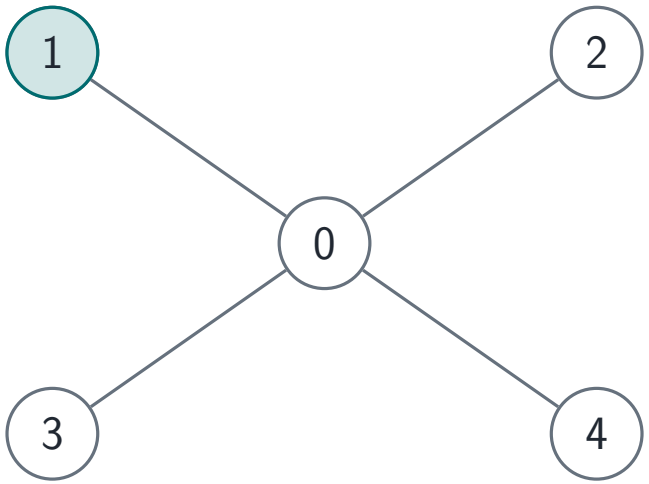
Delete v *and all its neighbors*.

$$H \text{ has an independent set of size at least } r \text{ containing } v$$
$$\iff \quad H_v \text{ has an independent set of size at least } r - 1.$$

Proof: remove v in the forward direction; add v in the reverse direction.

IndSet-Threshold Search to IndSet-Threshold Dec

Example: find an independent set of size ≥ 4.

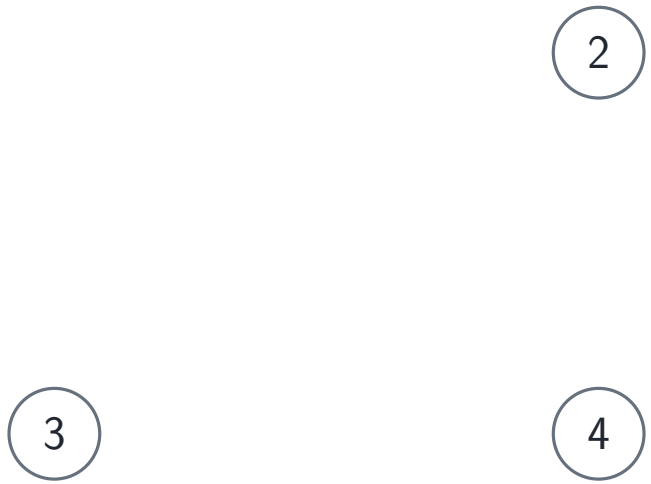


Try $v = 1$ first: To test whether we can include 1, remove it and its neighbor 0:

$$H_1 = G - \{0, 1\}.$$

IndSet-Threshold Search to IndSet-Threshold Dec

Example: find an independent set of size ≥ 4.



Try $v = 1$ first: To test whether we can include 1, remove it and its neighbor 0:

$$H_1 = G - \{0, 1\}.$$

H_1 contains the isolated vertices $2, 3, 4$.

$$\text{Dec}(H_1, 3) = \text{YES}.$$

Keep $S = \{1\}$ and seek two more vertices in H_1.

IndSet-Threshold Search to IndSet-Threshold Dec

```
1  if k = 0 then return ∅
2  if Dec(G, k) = NO then return ⊥
3  H ← G;   S ← ∅;   r ← k
4  while r > 0 do
5  |  Choose v ∈ V(H);   H_v ← H − ({v} ∪ N_H(v))
6  |  if Dec(H_v, r − 1) = YES then
   |  |
8  |  else
   |  |
10 return S
```

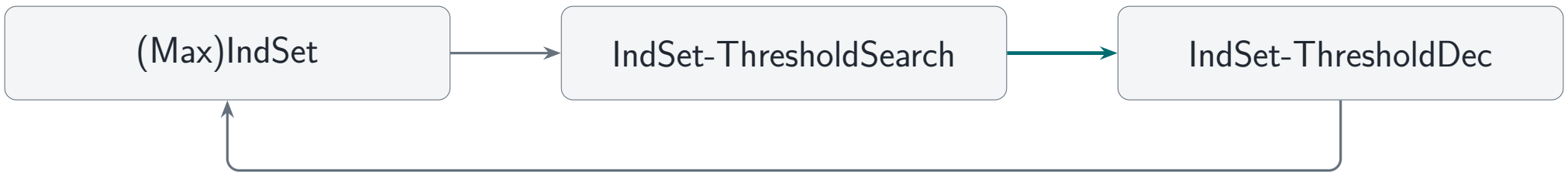
IndSet-Threshold Search to IndSet-Threshold Dec

```
1  if k = 0 then return ∅
2  if Dec(G, k) = NO then return ⊥
3  H ← G;   S ← ∅;   r ← k
4  while r > 0 do
5  |  Choose v ∈ V(H);   H_v ← H − ({v} ∪ N_H(v))
6  |  if Dec(H_v, r − 1) = YES then
7  |  |  S ← S ∪ {v};   H ← H_v;   r ← r − 1
8  |  else
   |  |
10 return S
```

IndSet-Threshold Search to IndSet-Threshold Dec

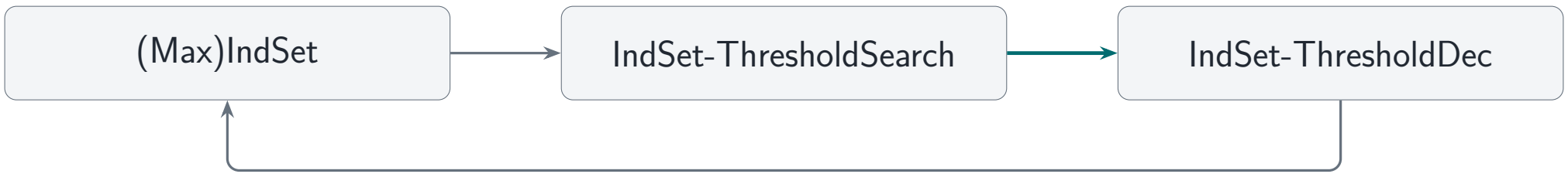
if $k = 0$ **then return** $\varnothing$
if $\mathrm{Dec}(G, k) = \mathrm{NO}$ **then return** $\perp$
$H \leftarrow G; \quad S \leftarrow \varnothing; \quad r \leftarrow k$
while $r > 0$ **do**
    Choose $v \in V(H); \quad H_v \leftarrow H - (\{v\} \cup N_H(v))$
    if $\mathrm{Dec}(H_v, r-1) = \mathrm{YES}$ **then**
        $S \leftarrow S \cup \{v\}; \quad H \leftarrow H_v; \quad r \leftarrow r - 1$
    else
        $H \leftarrow H - \{v\}$
return S

IndSet-Threshold Search to IndSet-Threshold Search



$A \longrightarrow B$: Reduce A to B.

IndSet-Threshold Search to IndSet-Threshold Search



$A \longrightarrow B$: Reduce A to B.

All the reductions run in polynomial time; although the last reduction is the slowest and makes $O(n)$ oracle calls.

Interval Scheduling- Opt

Input: closed intervals $[a_0, b_0], \ldots, [a_{n-1}, b_{n-1}]$, with rational endpoints and $a_i \leq b_i$.

Interval Scheduling- Opt

Input: closed intervals $[a_0, b_0], \ldots, [a_{n-1}, b_{n-1}]$, with rational endpoints and $a_i \leq b_i$.
Output: Choose as many pairwise disjoint intervals as possible.

$$S \subseteq \{0, \ldots, n-1\} \quad \text{of maximum size,}$$
$$[a_i, b_i] \cap [a_j, b_j] = \varnothing \quad \text{for all distinct } i, j \in S.$$

Interval Scheduling- Opt

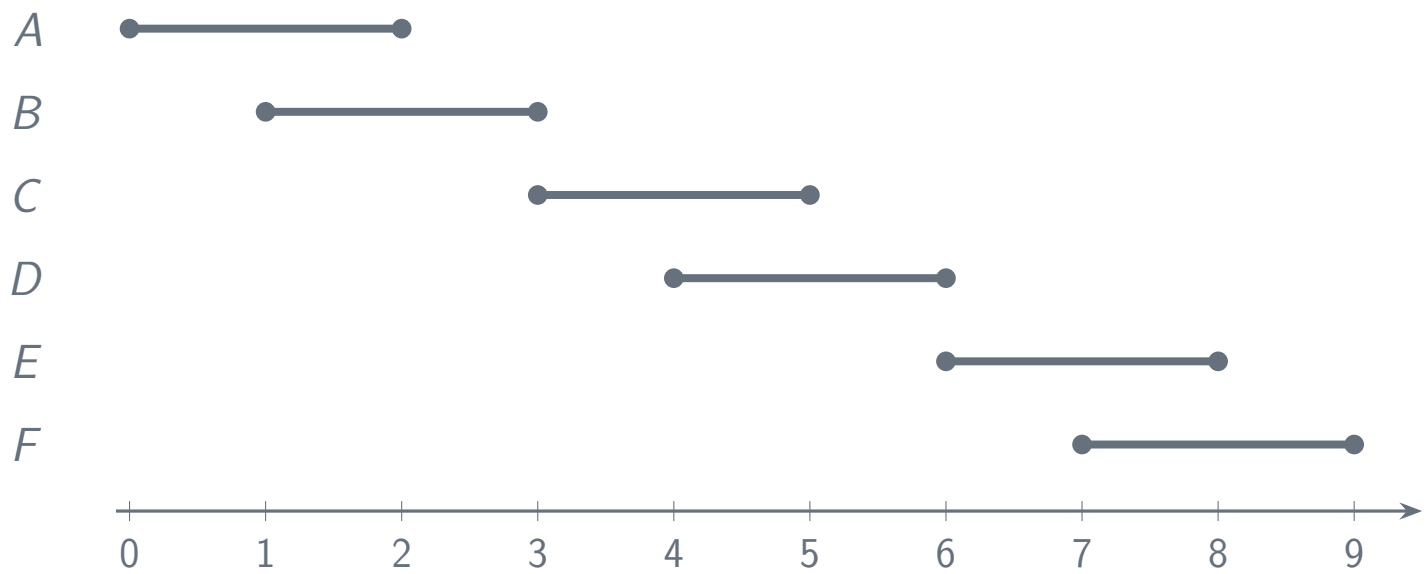
Input: closed intervals $[a_0, b_0], \ldots, [a_{n-1}, b_{n-1}]$, with rational endpoints and $a_i \leq b_i$.
Output: Choose as many pairwise disjoint intervals as possible.

$$S \subseteq \{0, \ldots, n-1\} \quad \text{of maximum size,}$$
$$[a_i, b_i] \cap [a_j, b_j] = \varnothing \quad \text{for all distinct } i, j \in S.$$

Q: Why rational endpoints?
Q: Which version have you seen before?

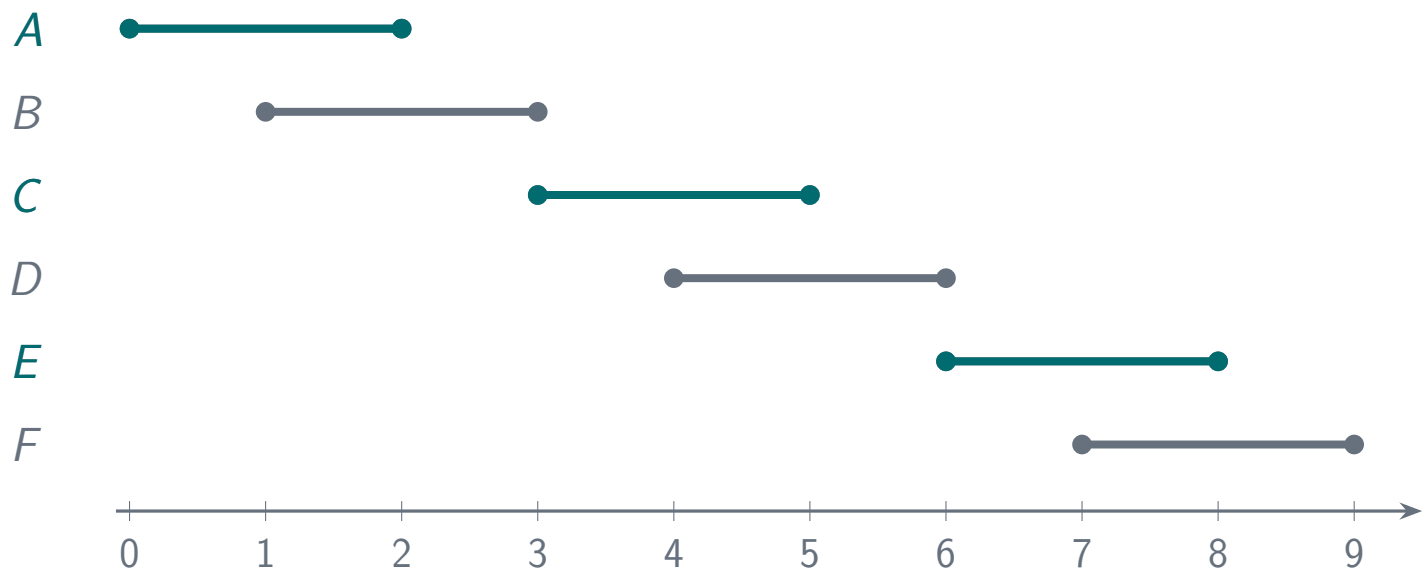
Interval Scheduling: Example

Choose a largest collection of disjoint intervals.



Interval Scheduling: Example

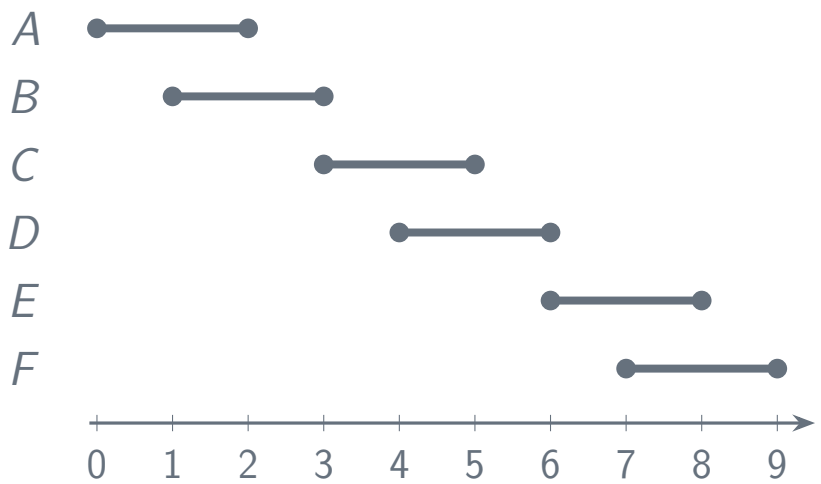
Choose a largest collection of disjoint intervals.



One solution is $\{A, C, E\}$, of size 3.

Interval Scheduling and Independent Set

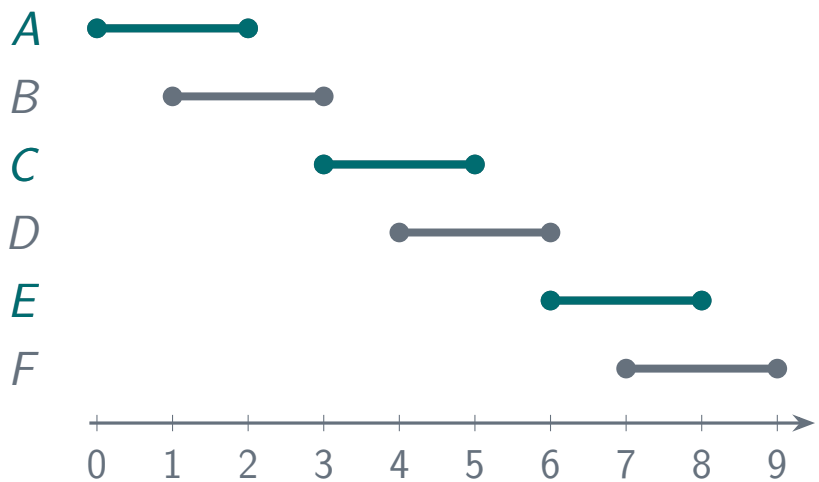
Represent each interval by a vertex; join two vertices exactly when their intervals intersect.





Interval Scheduling and Independent Set

Represent each interval by a vertex; join two vertices exactly when their intervals intersect.

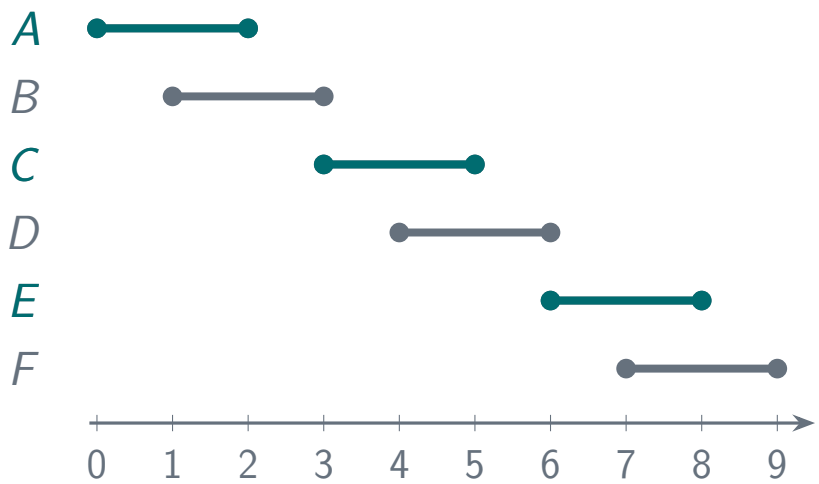




Disjoint intervals correspond exactly to independent vertices.

Interval Scheduling and Independent Set

Represent each interval by a vertex; join two vertices exactly when their intervals intersect.





Disjoint intervals correspond exactly to independent vertices. The conflict graphs from intervals have additional structure that the scheduling algorithm can use.

Greedy Algorithm for Interval Scheduling

Process intervals in increasing order of end time.

1 Sort the intervals so that $b'_0 \leq b'_1 \leq \cdots \leq b'_{n-1}$
2 $S \leftarrow \varnothing$
3 **for** $i = 0, \ldots, n-1$ **do**
4 | **if** **then**
5 | | $S \leftarrow S \cup \{i\}$
6 **return** S

Greedy Algorithm for Interval Scheduling

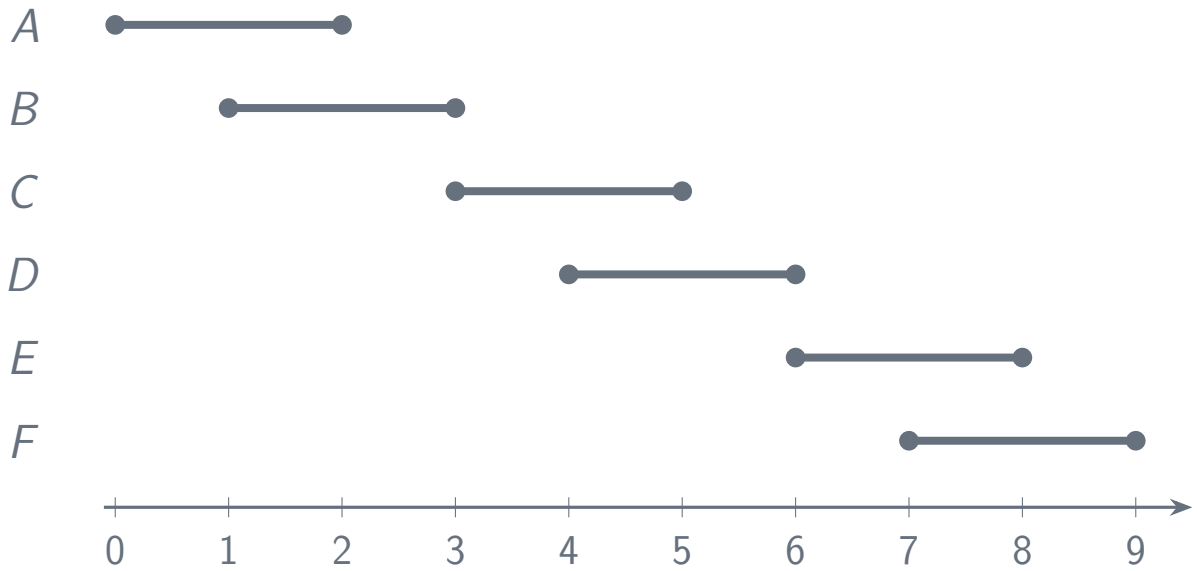
Process intervals in increasing order of end time.

1 Sort the intervals so that $b'_0 \leq b'_1 \leq \cdots \leq b'_{n-1}$
2 $S \leftarrow \varnothing$
3 **for** $i = 0, \ldots, n-1$ **do**
4 **if** $[a'_i, b'_i]$ *is disjoint from every selected interval* **then**
5 $S \leftarrow S \cup \{i\}$
6 **return** S

Always take the next available interval with earliest end time.

Greedy Algorithm: Example

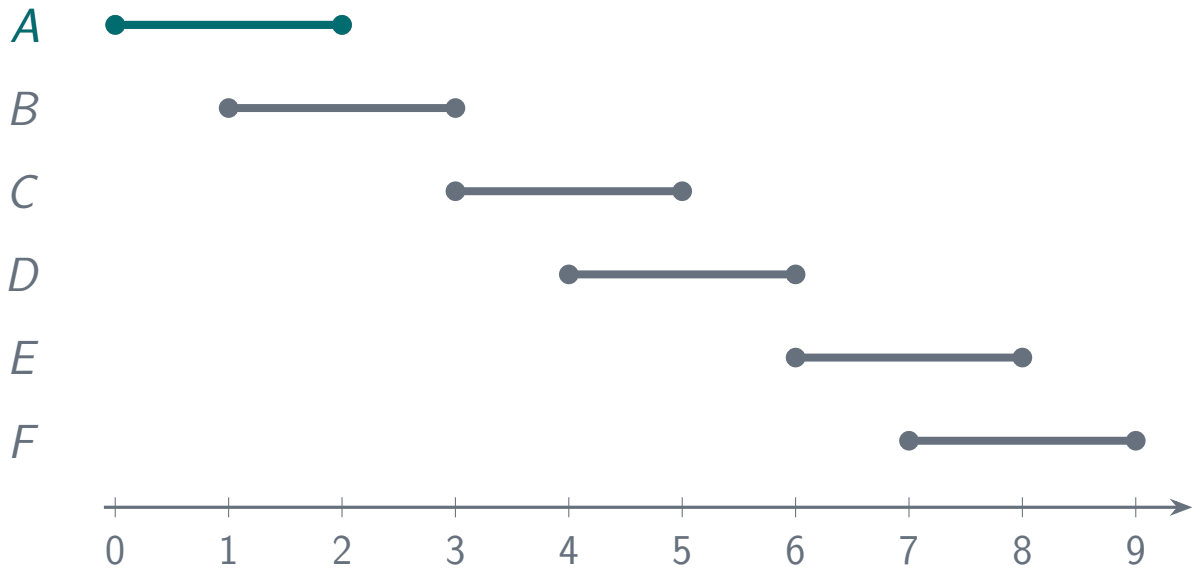
End-time order: A, B, C, D, E, F.



Start with $S = \varnothing$.

Greedy Algorithm: Example

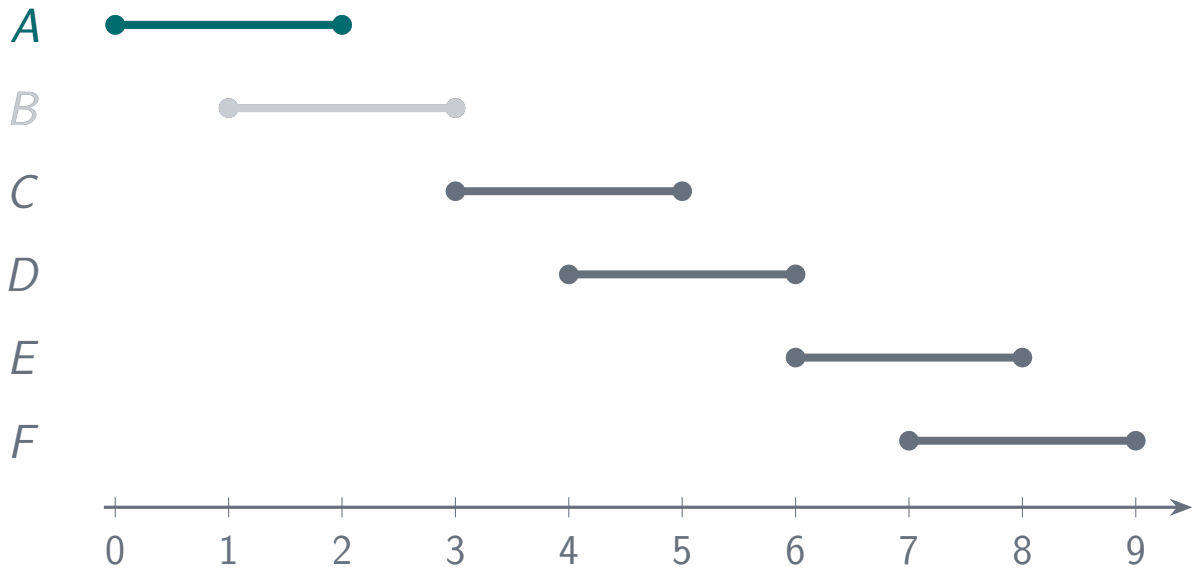
End-time order: A, B, C, D, E, F.



Select $A = [0, 2]$.

Greedy Algorithm: Example

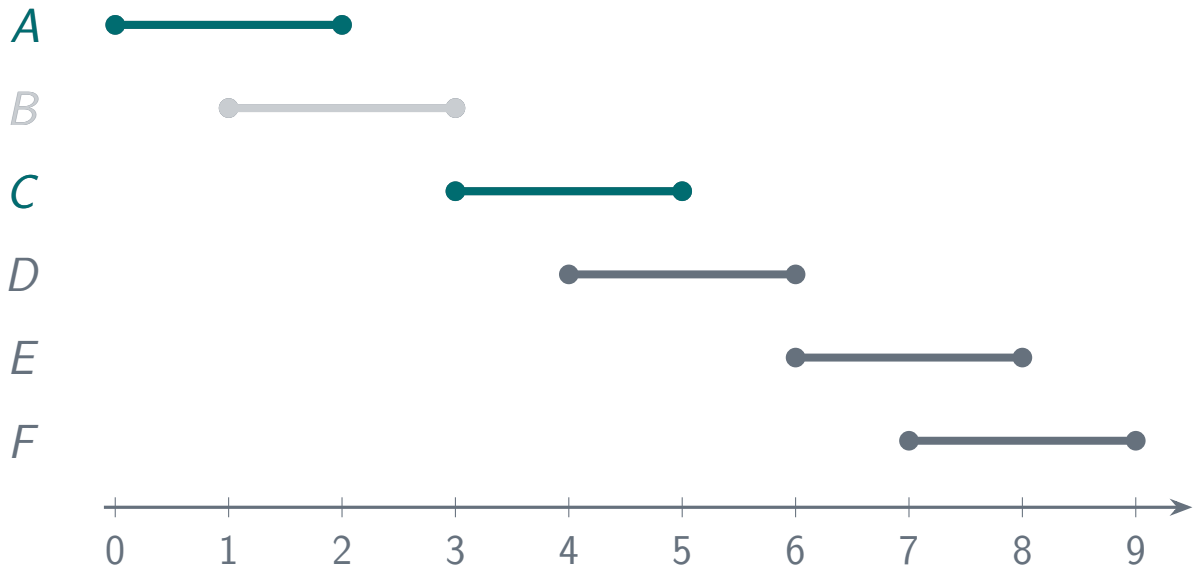
End-time order: A, B, C, D, E, F.



Reject B: it intersects A.

Greedy Algorithm: Example

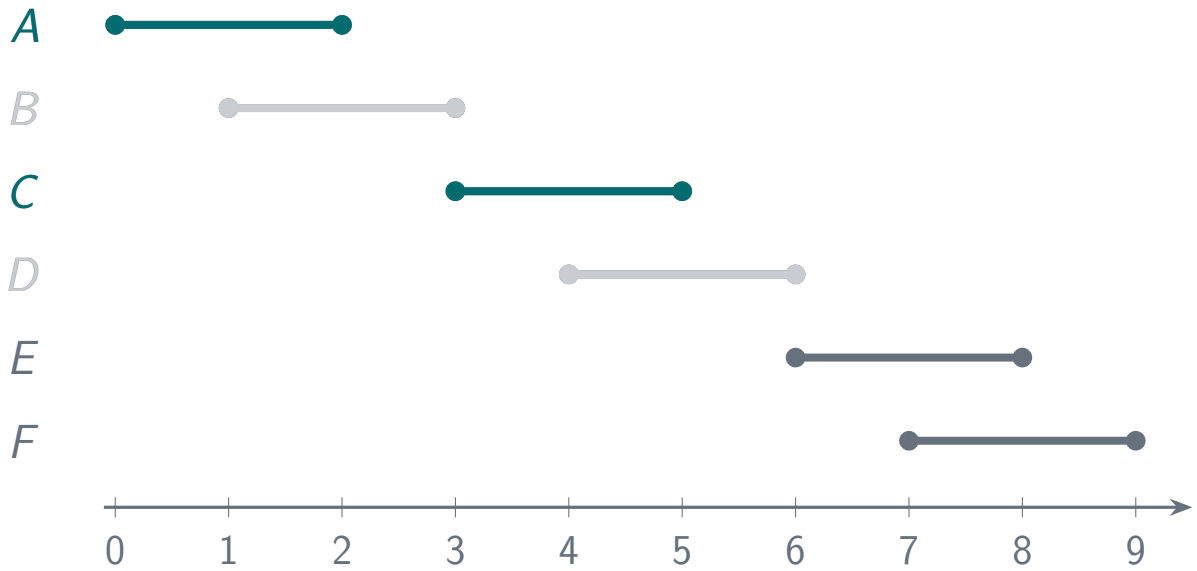
End-time order: A, B, C, D, E, F.



Select $C = [3, 5]$: $3 > 2$.

Greedy Algorithm: Example

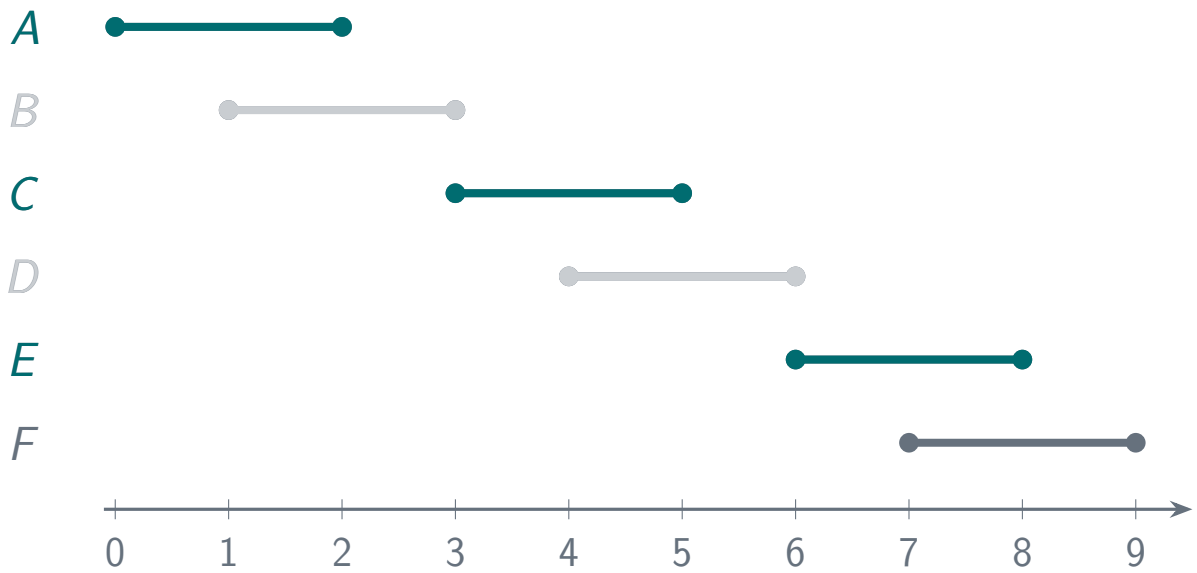
End-time order: A, B, C, D, E, F.



Reject D: it intersects C.

Greedy Algorithm: Example

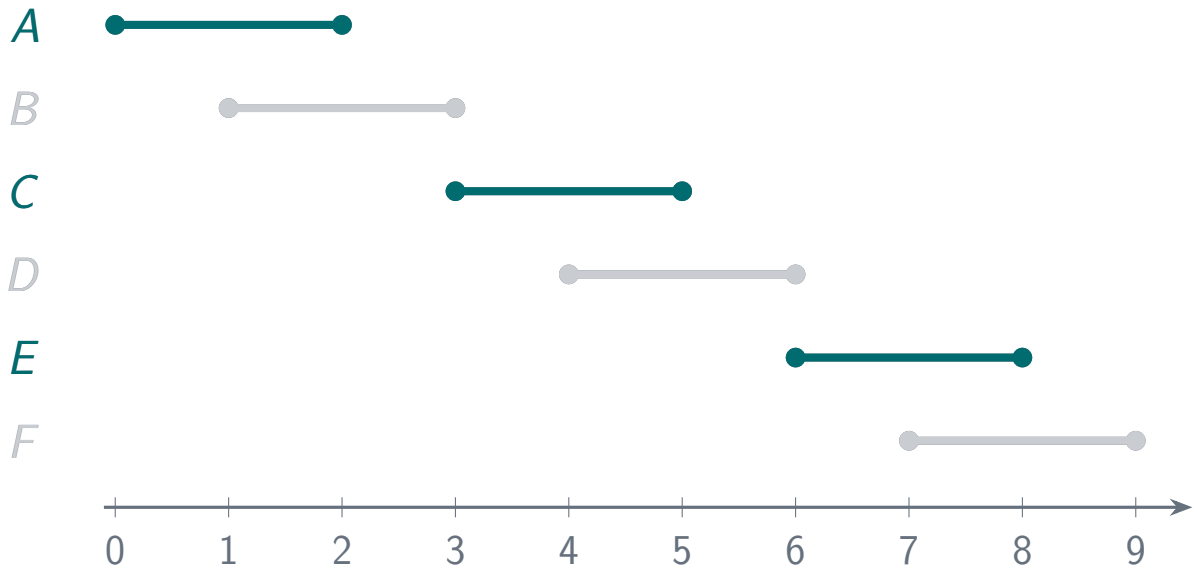
End-time order: A, B, C, D, E, F.



Select $E = [6, 8]$: $6 > 5$.

Greedy Algorithm: Example

End-time order: A, B, C, D, E, F.



Reject F: it intersects E.

Return $\{A, C, E\}$.

Interval Scheduling: Optimality

Theorem. The greedy algorithm finds a maximum-sized disjoint subset.

Interval Scheduling: Optimality

Theorem. The greedy algorithm finds a maximum-sized disjoint subset. Let $g_1, \ldots, g_k$ be the intervals found by the greedy algorithm and $o_1, \ldots, o_{k^*}$ an optimal solution.

Interval Scheduling: Optimality

Theorem. The greedy algorithm finds a maximum-sized disjoint subset. Let $g_1, \ldots, g_k$ be the intervals found by the greedy algorithm and $o_1, \ldots, o_{k^*}$ an optimal solution. **Claim:**

greedy stays ahead. For every $1 \leq j \leq k^*$, g_j exists and

$$\text{end}(g_j) \leq \text{end}(o_j).$$

Interval Scheduling: Optimality

Theorem. The greedy algorithm finds a maximum-sized disjoint subset. Let $g_1, \ldots, g_k$ be the intervals found by the greedy algorithm and $o_1, \ldots, o_{k^*}$ an optimal solution. **Claim:**

greedy stays ahead. For every $1 \leq j \leq k^*$, g_j exists and

$$\text{end}(g_j) \leq \text{end}(o_j).$$

Base case: g_1 has the earliest end time of all intervals.

Interval Scheduling: Proof Picture

Replace the optimal intervals one at a time by the greedy choices.

(B, D, F)



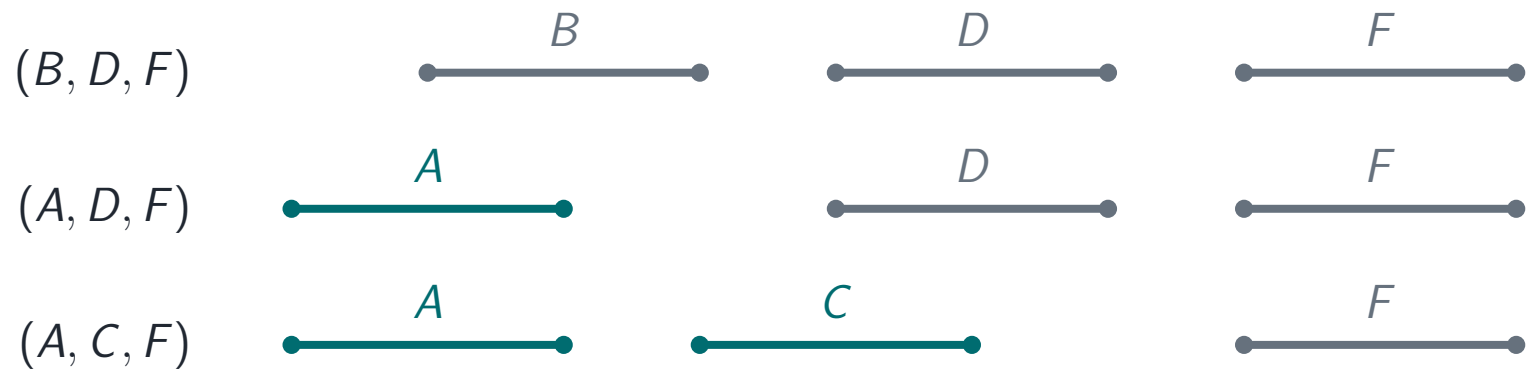
Interval Scheduling: Proof Picture

Replace the optimal intervals one at a time by the greedy choices.



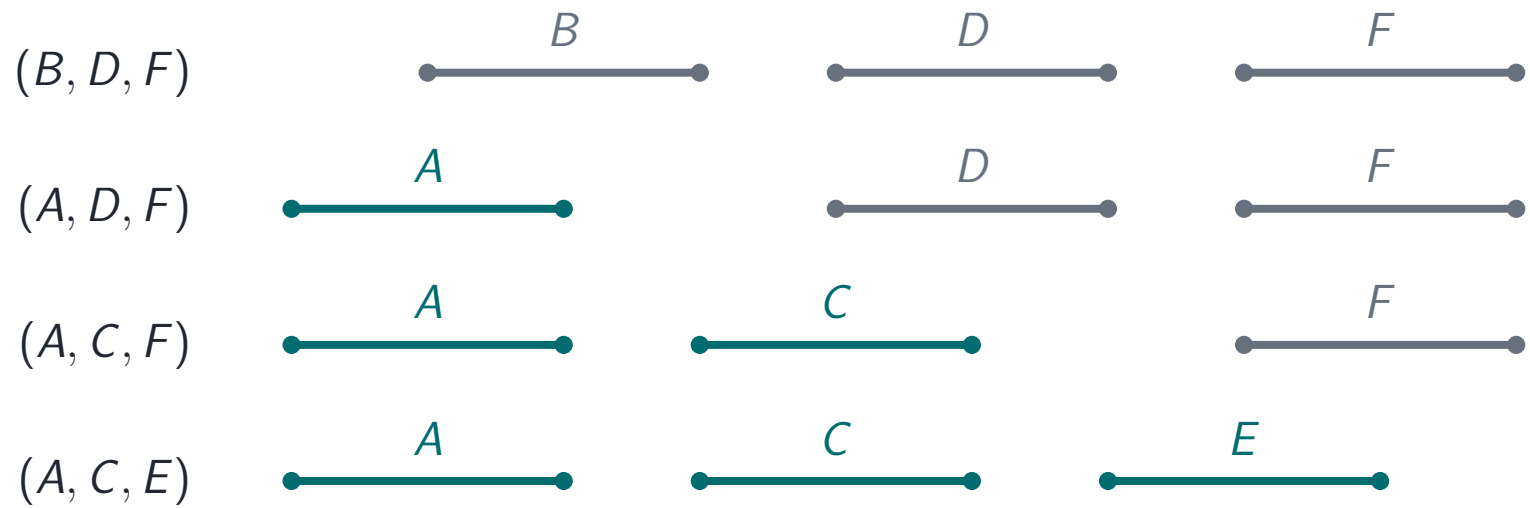
Interval Scheduling: Proof Picture

Replace the optimal intervals one at a time by the greedy choices.



Interval Scheduling: Proof Picture

Replace the optimal intervals one at a time by the greedy choices.



Each replacement ends earlier and leaves the later intervals available.

Interval Scheduling: Inductive Step

Assume g_j exists and $\text{end}(g_j) \leq \text{end}(o_j)$, with $j < k^*$.

Interval Scheduling: Inductive Step

Assume g_j exists and $\text{end}(g_j) \leq \text{end}(o_j)$, with $j < k^*$. Since the optimal intervals are disjoint,

$$\text{end}(g_j) \leq \text{end}(o_j) < \text{start}(o_{j+1}).$$

Interval Scheduling: Inductive Step

Assume g_j exists and $\text{end}(g_j) \leq \text{end}(o_j)$, with $j < k^*$. Since the optimal intervals are disjoint,

$$\text{end}(g_j) \leq \text{end}(o_j) < \text{start}(o_{j+1}).$$

Thus o_{j+1} is still available after choosing g_j.

Interval Scheduling: Inductive Step

Assume g_j exists and $\text{end}(g_j) \leq \text{end}(o_j)$, with $j < k^*$. Since the optimal intervals are disjoint,

$$\text{end}(g_j) \leq \text{end}(o_j) < \text{start}(o_{j+1}).$$

Thus o_{j+1} is still available after choosing g_j. Greedy takes an available interval with the earliest end time. Therefore,

$$g_{j+1} \text{ exists}, \qquad \text{end}(g_{j+1}) \leq \text{end}(o_{j+1}).$$

Interval Scheduling: Concluding the Proof

The claim holds for every $j \leq k^*$. In particular, the greedy algorithm selects at least k^* intervals:

$$k \geq k^*.$$

Interval Scheduling: Concluding the Proof

The claim holds for every $j \leq k^*$. In particular, the greedy algorithm selects at least k^* intervals:

$$k \geq k^*.$$

The greedy output is itself a disjoint collection. Optimality of k^* gives

$$k \leq k^*.$$

Interval Scheduling: Concluding the Proof

The claim holds for every $j \leq k^*$. In particular, the greedy algorithm selects at least k^* intervals:

$$k \geq k^*.$$

The greedy output is itself a disjoint collection. Optimality of k^* gives

$$k \leq k^*.$$

Hence $k = k^*$: the greedy algorithm is optimal.

Interval Scheduling: Running Time

- Sort by increasing end time: $O(n \log n)$.

Interval Scheduling: Running Time

- Sort by increasing end time: $O(n \log n)$.
- During the scan, remember the end of the last selected interval. Accept $[a'_i, b'_i]$ exactly when

$$S = \varnothing \quad \text{or} \quad a'_i > \text{lastEnd}.$$

Interval Scheduling: Running Time

- Sort by increasing end time: $O(n \log n)$.
- During the scan, remember the end of the last selected interval. Accept $[a'_i, b'_i]$ exactly when

$$S = \varnothing \quad \text{or} \quad a'_i > \text{lastEnd}.$$

$$\text{Total time} = O(n \log n) + O(n) = O(n \log n).$$

Interval Scheduling: Running Time

- Sort by increasing end time: $O(n \log n)$.
- During the scan, remember the end of the last selected interval. Accept $[a'_i, b'_i]$ exactly when

$$S = \varnothing \quad \text{or} \quad a'_i > \text{lastEnd}.$$

$$\text{Total time} = O(n \log n) + O(n) = O(n \log n).$$

Q: How would the algorithm change if you wanted to sort with start time?